

Benchmarking Database Instances

To properly analyze performance of a given cloud instance with Postgres, multiple benchmarking tools and approaches should be combined:

- synthetic benchmarks for Postgres server (pgbench, sysbench)
- “real workload” benchmarks (pgreplay), optionally with deep SQL query analysis (nancy)
- disk IO benchmarks (fio, bonnie++, seek-scaling)
- network benchmark (iperf3)

During benchmarking, any heavy workload not related to the benchmark itself must be excluded (verify with `top`, `dstat -f`).

Synthetic benchmark using pgbench

Running four synthetic benchmarks is recommended:

- small-sized (less than `shared_buffers`), SELECTs only
- small-sized, mixed workload
- large scale (data volumes are larger than `shared_buffers` and RAM), SELECTs only
- large scale, mixed workload

In the first two cases, the data will remain in shared buffers, so disk reads are not supposed to be involved, so only CPU and RAM will be tested. In the latter two cases, the benchmark will involve disk operations, so whole system (excluding network) will be analyzed.

Small-sized, SELECTs only

Example of running a series of `pgbench` benchmarks on an instance with Postgres installed:

```
s=1700
N=300
psql postgres -c "drop database if exists pgbench_s${s};"
psql postgres -c "create database pgbench_s${s};"
/opt/gitlab/embedded/bin/pgbench -i -s "$s" "pgbench_s${s}"
rm "results_s${s}_selects".bench
for i in $(seq "$N"); do
  j="$i"
  c="$i"
  res=$( \
    /opt/gitlab/embedded/bin/pgbench "pgbench_s${s}" \
      -h /var/opt/gitlab/postgresql \
      -T 30 -j "$j" -M prepared -c "$c" --select-only \
    | tail -n1
  )
```

```

echo "$c $res" >> "results_s${s}_selects.bench"
echo "-c $c -j $j --> $res"
done

```

Notes:

- for N=300, whole run will take several hours, so it is recommended to use `tmux`
- the scale in the code above is 1700 (`-s 1700`), this will give ~25GB of data in the `pgbench` database; compare it with the value `shared_buffers`
- the code runs 300 tests, increasing the number of concurrent sessions from 1 to 300
- each run lasts 30 seconds (`-T 30`)
- in the code above, prepared statements are being used (`-M prepared`)
- `pgbench`'s output contains two TPS (transactions per second) numbers: “including connections establishing” and “excluding connections establishing”; since this benchmark is to be run without involving network, the numbers should be very close; only “excluding connections establishing” number is being saved to the `results_selects.bench` file (notice `| tail -n1`)

Small-sized, mixed workload

Given the initialized `pgbench` done in the previous step:

```

rm "results_s${s}_mixed".bench
for i in $(seq "$N"); do
  j="$i"
  c="$i"
  res=$( \
    /opt/gitlab/embedded/bin/pgbench "pgbench_s${s}" \
    -h /var/opt/gitlab/postgresql \
    -T 30 -j "$j" -M prepared -c "$c" \
    | tail -n1
  )
  echo "$c $res" >> "results_s${s}_mixed.bench"
  echo "-c $c -j $j --> $res"
done

```

Notes:

- the only difference with the previous step is that `-S` option is excluded

Larger scale, SELECTs only

Same as “Small-sized, SELECTs only”, but the `pgbench` database must be initialized using larger scale. For instance, `-s 100000` will produce ~1.5TB of data.

Larger scale, mixed workload

The code is the same as in step “Small-sized, mixed workload”

Visualizing results with gnuplot

In this example the benchmark results for two instances are combined in one picture:

```
N=300
for s in 1700 100000; do
  for workload in "selects" "mixed"; do
    fname="s${s}_${workload}"
    echo "Processing fname: $fname..."
    scp "nikolays@postgres-dbteam-01-db-gstg.c.gitlab-staging-1.internal:/home/nikolays/results_$fname.new.bench"
    scp "nikolays@postgres-dbteam-01.db.stg.gitlab.com:/home/nikolays/results_$fname.bench"
    gnuplot << EOF
set terminal png size 500,500
set size 1, 1
set output 'bench_${fname}.png'
set title "`date '+%Y-%m-%d %H:%M'`, Workload: $workload\npgbench -j{1..$N} -c{1..$N}, Scale"
set key left top
set grid y
set xlabel '# of clients'
set ylabel 'TPS (excl. conn)'
set datafile separator ' '
plot 'results_${fname}.old.bench' using 4 with lines title 'old server (Azure)', \
      'results_${fname}.new.bench' using 4 with lines title 'new server (GCP)'
EOF
    open "bench_${fname}.png" || true
  done
done
```

Notes:

- if all files are present on remote hosts, 4 pictures will be created and opened

File system benchmark

```
sudo fio --randrepeat=1 --ioengine=libaio --direct=1 --gtod_reduce=1 \
  --name=test --filename=test --bs=4k --iodepth=64 --size=4G \
  --readwrite=randrw --rwmixread=75 \
&& sudo rm ./test
```